

ADO.NET • BOOTSTRAP • AJAX • ASP.NET MVC

Study Notes, Examples & Question–Answer Bank

Simple explanations for college study and exam revision

Reference note: The supplied .NET PDF is a scanned document. Its text is not reliably readable/extractable, so these notes are a structured beginner study guide for the four requested topics, not a page-by-page transcription of the reference.

How to use these notes

Study the topic outline first, then read each explanation and run through the examples. The code is illustrative and may need small changes depending on your .NET version, project template, database provider, and installed packages.

Contents

- 1. ADO.NET — architecture, objects, connection, commands, readers, adapters, DataSet, CRUD, parameters and transactions
- 2. Bootstrap — overview, setup, containers, grid, typography, buttons, forms, tables and responsive utilities
- 3. AJAX — asynchronous requests, XMLHttpRequest, jQuery AJAX, JSON, advantages and limitations
- 4. ASP.NET MVC — MVC pattern, routing, controllers, actions, models, views, Razor, validation and CRUD flow
- 5. Question–Answer Bank — short and descriptive questions with answers

1. ADO.NET

Meaning: ADO.NET is a .NET data-access technology used by applications to connect to data sources such as SQL Server, execute commands, and retrieve or update data.

1.1 Topics and subtopics

- Introduction and purpose of ADO.NET
- Connected and disconnected architecture
- Main objects: Connection, Command, DataReader, DataAdapter and DataSet
- Connection string
- ExecuteNonQuery(), ExecuteScalar() and ExecuteReader()
- Parameterized queries and CRUD operations
- Transactions and exception handling

1.2 ADO.NET architecture

ADO.NET provides two common ways to work with data:

- **Connected model:** The application keeps a connection open while it reads data, commonly using DataReader. It is fast and forward-only, but the connection remains occupied during reading.
- **Disconnected model:** Data is fetched into memory, commonly a DataSet or DataTable, and the connection can be closed. A DataAdapter can fill the in-memory data and later send changes back.

Typical flow: Application → Connection → Command → Database. Results can be read directly with a DataReader or filled into a DataSet/DataTable through a DataAdapter.

1.3 Important ADO.NET objects

Connection: Opens and manages a connection to the database. Example: SqlConnection.

Command: Represents an SQL statement or stored procedure. Example: SqlCommand.

DataReader: Reads rows sequentially, usually while the connection is open. Example: SqlDataReader.

DataAdapter: Bridges a database and a DataSet/DataTable; it can fill data and update changes.

DataSet: An in-memory, disconnected collection of tables and relations.

DataTable: An in-memory table of rows and columns.

1.4 Connection string

A connection string tells the application where the database is and how to authenticate. In this example, **Data Source=.** means the local SQL Server instance, **Initial Catalog=CollegeDB** selects the database, and **Integrated Security=True** uses the current Windows identity instead of a username and password.

```
string conStr =  
    "Data Source=.;Initial Catalog=CollegeDB;" +  
    "Integrated Security=True;";
```

The exact server name and authentication options depend on the SQL Server installation. Keep credentials out of source code in real applications.

1.5 Open a database connection

```
using Microsoft.Data.SqlClient;  
  
string conStr = "Data Source=.;Initial Catalog=CollegeDB;Integrated Security=True;TrustServerCertificate=True;";  
  
using (SqlConnection con = new SqlConnection(conStr))  
{  
    con.Open();  
    Console.WriteLine("Connected successfully");  
}
```

The **using** statement disposes the connection automatically, including when an exception occurs. Add the `Microsoft.Data.SqlClient` package to projects that use this provider.

1.6 ExecuteNonQuery()

`ExecuteNonQuery()` is used for SQL statements that do not return a result set, such as INSERT, UPDATE and DELETE. It returns the number of affected rows.

```
string sql = "INSERT INTO Students(Name) VALUES (@name)";  
using var con = new SqlConnection(conStr);  
using var cmd = new SqlCommand(sql, con);  
cmd.Parameters.AddWithValue("@name", "Asha");  
con.Open();  
int rows = cmd.ExecuteNonQuery();  
Console.WriteLine(rows + " row(s) inserted");
```

1.7 ExecuteScalar()

`ExecuteScalar()` returns the first column of the first row. It is useful for a single value, such as COUNT, SUM or a newly generated ID.

```
string sql = "SELECT COUNT(*) FROM Students";  
using var con = new SqlConnection(conStr);  
using var cmd = new SqlCommand(sql, con);  
con.Open();  
int total = Convert.ToInt32(cmd.ExecuteScalar());
```

1.8 ExecuteReader()

`ExecuteReader()` runs a query and returns a `DataReader`. `Read()` advances to the next row.

```
string sql = "SELECT Id, Name FROM Students";  
using var con = new SqlConnection(conStr);  
using var cmd = new SqlCommand(sql, con);  
con.Open();  
using SqlDataReader reader = cmd.ExecuteReader();  
while (reader.Read())  
{  
    Console.WriteLine(reader["Id"] + " " + reader["Name"]);  
}
```

1.9 Parameterized queries

Parameters pass values separately from SQL text. They help prevent SQL injection and handle values safely. Do not build SQL by joining user input into the query string.

```
string sql = "SELECT Id, Name FROM Students WHERE Id = @id";
using var con = new SqlConnection(conStr);
using var cmd = new SqlCommand(sql, con);
cmd.Parameters.AddWithValue("@id", 5);
con.Open();
using var reader = cmd.ExecuteReader();
```

1.10 DataAdapter and DataSet

A DataAdapter can retrieve rows into a DataTable or DataSet. The data can then be used after the database connection is closed.

```
string sql = "SELECT Id, Name FROM Students";
using var con = new SqlConnection(conStr);
using var da = new SqlDataAdapter(sql, con);
DataTable table = new DataTable();
da.Fill(table);
// table now contains the retrieved rows
```

1.11 CRUD summary

- **Create:** INSERT; commonly ExecuteNonQuery().
- **Read:** SELECT; commonly ExecuteReader(), DataAdapter or ExecuteScalar().
- **Update:** UPDATE; commonly ExecuteNonQuery().
- **Delete:** DELETE; commonly ExecuteNonQuery().

1.12 Transactions and error handling

A transaction groups database operations so they can be committed together or rolled back together. Use it when multiple changes must succeed or fail as one unit.

```
using var con = new SqlConnection(conStr);
con.Open();
using var tx = con.BeginTransaction();
try
{
    using var cmd = new SqlCommand(
        "UPDATE Accounts SET Balance = Balance - @amount WHERE Id = @id", con, tx);
    cmd.Parameters.AddWithValue("@amount", 100);
    cmd.Parameters.AddWithValue("@id", 1);
    cmd.ExecuteNonQuery();
    tx.Commit();
}
catch
{
    tx.Rollback();
    throw;
}
```

2. Bootstrap

Meaning: Bootstrap is a front-end framework containing ready-made CSS classes and JavaScript components for building responsive, mobile-first web pages.

2.1 Topics and subtopics

- Introduction and features
- Adding Bootstrap to a project
- Containers and responsive breakpoints
- 12-column grid system
- Typography and colors

- Buttons, tables and forms
- Navbar and common components
- Responsive utility classes

2.2 Features

- Responsive layout for different screen sizes
- Reusable styles and components
- Grid system based on 12 columns
- Consistent appearance across pages
- Can be used with plain HTML and many server-side frameworks

2.3 Add Bootstrap

Bootstrap can be included through a CDN link or downloaded into the project. Example using a CDN (use the version approved for your project):

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
      rel="stylesheet">
```

For JavaScript-powered components, include the Bootstrap bundle script before the closing body tag:

```
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
```

2.4 Containers

A `.container` provides a responsive fixed-width container at different breakpoints. A `.container-fluid` takes the full available width.

```
<div class="container">
  <h1>College Portal</h1>
  <p>Content inside a responsive container.</p>
</div>

<div class="container-fluid">
  Full-width content
</div>
```

2.5 Grid system

Bootstrap's grid uses rows and columns. A row contains columns; the 12-column layout lets you control how much width each column occupies. For example, `col-md-6` gives each column half the row width from the medium breakpoint upward.

```
<div class="container">
  <div class="row">
    <div class="col-md-6">Left half</div>
    <div class="col-md-6">Right half</div>
  </div>
</div>
```

Common responsive prefixes: `col-sm-`, `col-md-`, `col-lg-`, `col-xl-` and `col-xxl-`. A column can stack on smaller screens and sit side-by-side on larger screens.

2.6 Buttons, tables and forms

```
<button class="btn btn-primary">Save</button>
<button class="btn btn-outline-secondary">Cancel</button>

<table class="table table-striped">
  <thead><tr><th>ID</th><th>Name</th></tr></thead>
  <tbody><tr><td>1</td><td>Asha</td></tr></tbody>
</table>

<label class="form-label" for="name">Name</label>
<input class="form-control" id="name" type="text">
```

Common classes include `.btn`, `.btn-primary`, `.table`, `.table-striped`, `.form-label` and `.form-control`. These classes style elements; they do not by themselves save data or validate it on the server.

2.7 Responsive utilities

Bootstrap utility classes can change spacing, display, alignment and visibility at different breakpoints. For example, `d-none d-md-block` hides an element on small screens and displays it from the medium breakpoint upward.

```
<div class="d-none d-md-block">
  Visible on medium and larger screens
</div>

<div class="mt-3 p-2 text-center">
  Spacing and text alignment utilities
</div>
```

3. AJAX

Meaning: AJAX stands for Asynchronous JavaScript and XML. It is a technique for sending and receiving data from a server without reloading the entire web page. Modern applications commonly exchange JSON rather than XML.

3.1 Topics and subtopics

- Meaning and working of AJAX
- Asynchronous request lifecycle
- XMLHttpRequest
- jQuery \$.ajax()
- JSON data exchange
- AJAX with a server endpoint
- Advantages and limitations

3.2 How AJAX works

- The user performs an action, such as clicking a button.
- JavaScript sends a request to a server endpoint in the background.
- The server processes the request and returns data, often JSON.
- JavaScript handles the response and updates part of the page.

AJAX does not mean that a page never reloads; it means the request can be asynchronous and the page can be updated without a full reload.

3.3 XMLHttpRequest example

```
const xhr = new XMLHttpRequest();
xhr.open("GET", "/api/students", true);
xhr.onload = function () {
  if (xhr.status >= 200 && xhr.status < 300) {
    const students = JSON.parse(xhr.responseText);
    console.log(students);
  }
};
xhr.onerror = function () {
  console.log("Network error");
};
xhr.send();
```

The third argument `true` makes the request asynchronous. The response is parsed as JSON after a successful HTTP response.

3.4 jQuery AJAX example

```
$.ajax({
  url: "/api/students",
  method: "GET",
  dataType: "json",
  success: function (data) {
    console.log(data);
  },
  error: function () {
    console.log("Request failed");
  }
});
```

jQuery's \$.ajax() accepts options such as url, method, data, dataType, success and error. This example requires jQuery to be loaded.

3.5 Sending data with AJAX

```
$.ajax({
  url: "/api/students",
  method: "POST",
  contentType: "application/json",
  data: JSON.stringify({ name: "Asha" }),
  success: function (result) {
    console.log("Saved", result);
  }
});
```

A POST request commonly sends new data. The server must expose a matching endpoint and validate the submitted values. For cookie-authenticated applications, include the appropriate anti-forgery protection.

3.6 Advantages and limitations

- Advantages: partial page updates, less unnecessary page rendering, responsive user experience, and background data requests.
- Limitations: requires JavaScript, needs error/loading handling, can be affected by network delays, and must follow same-origin/CORS and security rules.

4. ASP.NET MVC

Meaning: ASP.NET MVC is a web application framework pattern that separates an application into Model, View and Controller responsibilities. In this guide, examples use the classic ASP.NET MVC style; ASP.NET Core MVC has some syntax and project differences.

4.1 Topics and subtopics

- MVC architecture and request flow
- Model, View and Controller
- Routing and URL mapping
- Controller and action methods
- Razor views and HTML helpers
- Passing data to a view
- Model binding and validation
- CRUD application flow

4.2 MVC architecture

Model: Represents application data and, commonly, validation/business rules.

View: Displays the user interface, usually HTML with Razor syntax.

Controller: Receives a request, coordinates work, and selects a response or view.

Typical flow: Browser request → Routing → Controller action → Model/data work → View rendering → HTML response to browser.

4.3 Controller and action

A controller is a class that handles requests. An action is a public method that responds to a route.

```
public class HomeController : Controller
{
    public ActionResult Index()
    {
        return View();
    }

    public ActionResult About()
    {
        ViewBag.Message = "About our college";
        return View();
    }
}
```

4.4 Routing

Routing maps an incoming URL to a controller and action. A common conventional route is {controller}/{action}/{id}. For example, /Student/Details/5 can map to the Details action of StudentController with id = 5.

```
routes.MapRoute(
    name: "Default",
    url: "{controller}/{action}/{id}",
    defaults: new { controller = "Home", action = "Index", id = UrlParameter.Optional }
);
```

4.5 Model and validation

```
public class Student
{
    public int Id { get; set; }

    [Required]
    public string Name { get; set; }

    [Range(1, 100)]
    public int Age { get; set; }
}
```

Data annotations such as [Required] and [Range] describe validation rules. A controller should check ModelState.IsValid before saving submitted data.

4.6 View and Razor

Razor lets a view mix HTML with C# expressions. The @model directive declares the model type expected by a strongly typed view.

```
@model Student

<h2>Student Details</h2>
<p>Id: @Model.Id</p>
<p>Name: @Model.Name</p>
<p>Age: @Model.Age</p>
```

4.7 Passing data from controller to view

A controller can pass data through a model (recommended for structured page data), ViewBag, or ViewData. ViewBag is a dynamic wrapper; ViewData is dictionary-based.

```
public ActionResult Welcome()
{
    ViewBag.Message = "Welcome to the portal";
    return View();
}

// In Welcome.cshtml:
<h2>@ViewBag.Message</h2>
```

4.8 Simple create action pattern

```
[HttpGet]
public ActionResult Create()
{
    return View();
}

[HttpPost]
[ValidateAntiForgeryToken]
public ActionResult Create(Student student)
{
    if (ModelState.IsValid)
    {
        // Save student using your data-access layer
        return RedirectToAction("Index");
    }
    return View(student);
}
```

The GET action displays the form. The POST action receives submitted form values through model binding, validates them, and then saves them. The save operation is represented by a comment because it depends on the application's database layer.

4.9 MVC advantages

- Separation of concerns makes code easier to organize.
- Views can be changed without placing UI code in data classes.
- Controllers provide a clear place to handle requests.
- Supports testable application design and URL routing.

5. Question–Answer Bank

Use these answers for quick revision. For long-answer questions, expand the points with the relevant example.

ADO.NET questions

Q. What is ADO.NET?

Answer: ADO.NET is a .NET data-access technology used to connect applications to databases, execute SQL commands, and retrieve or modify data.

Q. What is the difference between connected and disconnected architecture?

Answer: Connected architecture works with an open database connection, often using `DataReader`. Disconnected architecture stores retrieved data in memory, such as a `DataSet`, and does not require the connection to remain open while the data is used.

Q. Name five important ADO.NET objects.

Answer: `Connection`, `Command`, `DataReader`, `DataAdapter` and `DataSet` are important ADO.NET objects.

Q. What is the use of `SqlConnection`?

Answer: `SqlConnection` establishes and manages a connection between a .NET application and SQL Server.

Q. What is the use of `SqlCommand`?

Answer: `SqlCommand` represents an SQL query or stored procedure that the application executes against the database.

Q. Differentiate `ExecuteNonQuery()`, `ExecuteScalar()` and `ExecuteReader()`.

Answer: `ExecuteNonQuery()` is used for commands such as `INSERT`, `UPDATE` and `DELETE` and returns affected rows. `ExecuteScalar()` returns one value. `ExecuteReader()` returns a reader for reading rows.

Q. What is a DataReader?

Answer: A DataReader provides fast, forward-only access to query results while the database connection is open.

Q. What is a DataAdapter?

Answer: A DataAdapter acts as a bridge between a data source and a DataSet or DataTable. It can fill in-memory data and update the source.

Q. Why are parameterized queries used?

Answer: They separate SQL code from input values, help prevent SQL injection, and handle values more safely.

Q. What is a transaction?

Answer: A transaction groups database operations so they can be committed as a unit or rolled back if an error occurs.

Bootstrap questions

Q. What is Bootstrap?

Answer: Bootstrap is a front-end framework that provides CSS classes and JavaScript components for responsive, mobile-first websites.

Q. What is responsive web design?

Answer: Responsive design makes a web page adapt its layout to different screen sizes and devices.

Q. What is the Bootstrap grid system?

Answer: It is a responsive layout system based on rows and 12 columns. Column classes determine how much width an element uses at different breakpoints.

Q. Differentiate .container and .container-fluid.

Answer: .container uses responsive maximum widths at breakpoints, while .container-fluid occupies the full available width.

Q. What does col-md-6 mean?

Answer: It makes an element span six of twelve grid columns (half the row) at the medium breakpoint and above.

Q. Give examples of Bootstrap button classes.

Answer: .btn and .btn-primary create a styled primary button; .btn-outline-secondary creates an outlined secondary button.

Q. What is the use of .form-control?

Answer: It applies Bootstrap styling to form controls such as text inputs and textareas.

Q. How can Bootstrap be added to a webpage?

Answer: Bootstrap can be added using a CDN stylesheet link or by including the downloaded CSS files. JavaScript components may require the Bootstrap bundle.

AJAX questions

Q. What is AJAX?

Answer: AJAX is a web technique for sending and receiving data asynchronously so a page can update without a complete reload.

Q. What does AJAX stand for?

Answer: Asynchronous JavaScript and XML.

Q. Explain the working of AJAX.

Answer: JavaScript sends a background request to a server. The server processes it and returns data. JavaScript handles the response and updates part of the page.

Q. What is XMLHttpRequest?

Answer: XMLHttpRequest is a browser API used by JavaScript to send HTTP requests and receive server responses.

Q. What is JSON?

Answer: JSON (JavaScript Object Notation) is a text-based format commonly used to exchange structured data between a browser and a server.

Q. State two advantages of AJAX.

Answer: It can update part of a page without a full reload and can make the user experience feel more responsive.

Q. State two limitations of AJAX.

Answer: It depends on JavaScript and requires handling of network errors, security rules and asynchronous timing.

Q. What is the purpose of \$.ajax()?

Answer: \$.ajax() is a jQuery method for making configurable asynchronous HTTP requests.

ASP.NET MVC questions

Q. What is MVC?

Answer: MVC stands for Model–View–Controller. It separates data and rules, user interface, and request-handling logic.

Q. Explain Model, View and Controller.

Answer: Model represents application data and rules. View displays the user interface. Controller handles incoming requests and coordinates the response.

Q. What is routing in MVC?

Answer: Routing maps an incoming URL to a controller action and supplies route values such as an ID.

Q. What is a controller?

Answer: A controller is a class that handles incoming requests and returns a view or another action result.

Q. What is an action method?

Answer: An action method is a controller method that responds to a request, often returning an ActionResult.

Q. What is Razor?

Answer: Razor is a view syntax that allows C# expressions to be written within HTML using the @ symbol.

Q. What is model binding?

Answer: Model binding maps values from a request, such as form fields or route values, to action method parameters or model properties.

Q. What is ModelState.IsValid?

Answer: It indicates whether model binding and validation succeeded for the submitted model.

Q. Differentiate GET and POST in a form workflow.

Answer: GET is commonly used to display a form or retrieve data. POST is commonly used to submit data that may create or change server-side state.

Q. What is the purpose of ValidateAntiForgeryToken?

Answer: It checks an anti-forgery token on a request to help protect form submissions against cross-site request forgery.

Practice long-answer questions

- Explain ADO.NET architecture with connected and disconnected models.
- Explain the uses of Connection, Command, DataReader, DataAdapter and DataSet.

- Write a program to insert a record using parameterized SQL and ExecuteNonQuery().
- Explain Bootstrap's grid system with a suitable example.
- Explain the steps involved in an AJAX request.
- Explain MVC architecture and the request flow with a diagram.
- Explain model binding and validation in ASP.NET MVC.

Quick revision sheet

ADO.NET: Connection connects; Command executes; DataReader reads forward-only; DataAdapter fills/updates; DataSet stores disconnected data.

Bootstrap: container → row → columns; 12-column grid; responsive breakpoint classes; reusable CSS components.

AJAX: background request → server response → JavaScript updates part of page; JSON is commonly used.

MVC: Model = data/rules; View = UI; Controller = request handling; route maps URL to action.

Exam tip: For a 5-mark answer, write the definition, 3–4 key points, and a small example. For a 10-mark answer, add architecture/flow, advantages, and a complete example where relevant.